UNIT 3 MINI PROJECT

Problem Solving Component

1. Attendance Tracker

2. Sales Data Analyzer

Python programs built with lists and dictionaries, with complete source code, test scenarios and interpreted results.

NAME	Sriram Satvik Karanam
USN	25BTRAD049
BRANCH	CSE - AIDE (B Section)
COURSE	Python Programming

Overview

This report contains two complete Python mini projects. Both use only **lists** and **dictionaries** as the core data structures, are organised as classes with small reusable methods, validate every input, and ship with a menu-driven interface plus a separate test file that exercises many different scenarios.

Problem statements

Project	Problem statement	Required functionalities
1. Attendance Tracker	Python-based attendance tracker for a class using lists and dictionaries; store student names and attendance status, add or update records, display details, calculate each student's attendance percentage.	Store records; add students and update status; organised display; percentages and below-threshold students; test with different scenarios.
2. Sales Data Analyzer	Python-based sales data analyzer using lists and dictionaries to manage product names, quantities sold and prices; calculate total sales, find highest-selling product and summarise product-wise performance.	Store product details; add or update sales; total sales and product-wise revenue; highest seller and comparison; test with different datasets and interpret results.

Tools and environment

- Language: Python 3.12 (standard library only, no external packages needed to run the programs).
- Programs are in `attendance_tracker.py` and `sales_analyzer.py`; tests in `test_attendance.py` and `test_sales.py`.
- Every output shown in this report was produced by actually running the test files, not typed by hand.

How the report is organised

- Part A: Attendance Tracker (design, full code, 8 test scenarios with interpretation).
- Part B: Sales Data Analyzer (design, full code, 4 datasets plus update, error and empty-case tests).
- Part C: Conclusion and learning outcomes.

Part A: Attendance Tracker

Store attendance for a class, update it, display it and flag students with shortage.

A.1 Approach

Each student is a key in a dictionary. Its value is another dictionary mapping a class date to a one-letter status. A separate list keeps the students and the dates in insertion order so that tables always print in a predictable order.

Structure	Type	Purpose
students	list	Ordered student names (used for display order)
class_dates	list	Ordered dates on which a class was held
attendance	dict of dicts	{name: {date: status}} where status is P, A or L

Attendance rules

- **P** = Present, **A** = Absent, **L** = Late. A late arrival counts as attended.
- **Percentage** = (Present + Late) / Total classes recorded for that student x 100, rounded to 2 decimals.
- A student is flagged when percentage is **strictly below** the threshold (default 75%).
- Names are normalised with `.strip().title()` so 'rohan' and 'Rohan' refer to the same student.

A.2 Methods implemented

Method	What it does
add_student(name)	Adds a new student; rejects empty or duplicate names
add_class_date(date)	Registers a new class date; rejects duplicates
mark_attendance(name, date, status)	Creates a record or updates an existing one; reports 'added' or 'updated (A -> P)'
mark_whole_class(date, status_map)	Marks the full class in one call (unlisted students default to P)
counts(name) / percentage(name)	Counts P, L, A and returns the attendance percentage
below_threshold(threshold)	Returns students under the threshold, lowest first
display_records()	Prints the full grid of students vs dates with percentage
display_summary(threshold)	Prints P, L, A counts, percentage and OK / LOW status
class_average()	Average attendance percentage of the class

A.3 Source code: attendance_tracker.py

```
"""
Attendance Tracker  -  Python Programming (Unit 3 Mini Project)
Author : Sriram Satvik Karanam | USN : 25BTRAD049 | CSE-AIDE B

Data structures used
--------------------
students   : list   -> ordered list of student names
class_dates: list   -> ordered list of dates on which classes were held
attendance : dict   -> {student_name: {date: status}}
                       status is "P" (Present), "A" (Absent) or "L" (Late)
Late is counted as attended when percentages are calculated.
"""
```

```python
VALID_STATUS = {"P": "Present", "A": "Absent", "L": "Late"}
DEFAULT_THRESHOLD = 75.0


class AttendanceTracker:
    def __init__(self):
        self.students = []        # list of names
        self.class_dates = []     # list of dates (strings)
        self.attendance = {}      # dict of dicts

    # ------------------------------------------------------------
    # 1. Adding students and class dates
    # ------------------------------------------------------------
    def add_student(self, name):
        name = name.strip().title()
        if not name:
            raise ValueError("Student name cannot be empty.")
        if name in self.attendance:
            raise ValueError(f"Student '{name}' already exists.")
        self.students.append(name)
        self.attendance[name] = {}
        return name

    def add_class_date(self, date):
        date = date.strip()
        if not date:
            raise ValueError("Date cannot be empty.")
        if date in self.class_dates:
            raise ValueError(f"Class date '{date}' already exists.")
        self.class_dates.append(date)

    # ------------------------------------------------------------
    # 2. Marking and updating attendance
    # ------------------------------------------------------------
    def mark_attendance(self, name, date, status):
        """Add a new record OR update an existing one."""
        name = name.strip().title()
        status = status.strip().upper()
        if name not in self.attendance:
            raise KeyError(f"Student '{name}' not found.")
        if date not in self.class_dates:
            raise KeyError(f"Date '{date}' is not a recorded class date.")
        if status not in VALID_STATUS:
            raise ValueError("Status must be P, A or L.")
        old = self.attendance[name].get(date)
        self.attendance[name][date] = status
        return "added" if old is None else f"updated ({old} -> {status})"

    def mark_whole_class(self, date, status_map):
        """status_map = {name: status}; students not listed are marked P."""
        for name in self.students:
            self.mark_attendance(name, date, status_map.get(name, "P"))

    # ------------------------------------------------------------
    # 3. Calculations
    # ------------------------------------------------------------
    def counts(self, name):
        record = self.attendance[name]
        present = sum(1 for s in record.values() if s == "P")
        late = sum(1 for s in record.values() if s == "L")
        absent = sum(1 for s in record.values() if s == "A")
        return present, late, absent

    def percentage(self, name):
        name = name.strip().title()
        if name not in self.attendance:
            raise KeyError(f"Student '{name}' not found.")
        present, late, absent = self.counts(name)
        total = present + late + absent
        if total == 0:
            return 0.0
        return round((present + late) / total * 100, 2)

    def below_threshold(self, threshold=DEFAULT_THRESHOLD):
        """Return list of (name, percentage) below the threshold."""
```

```
        result = []
        for name in self.students:
            pct = self.percentage(name)
            if pct < threshold:
                result.append((name, pct))
        return sorted(result, key=lambda item: item[1])

    # ------------------------------------------------------------------
    # 4. Display
    # ------------------------------------------------------------------
    def display_records(self):
        if not self.students:
            print("No students recorded yet.")
            return
        name_w = max(len("Student"), max(len(s) for s in self.students)) + 2
        header = "Student".ljust(name_w)
        for d in self.class_dates:
            header += d.center(7)
        header += "  Att%".rjust(8)
        print(header)
        print("-" * len(header))
        for name in self.students:
            row = name.ljust(name_w)
            for d in self.class_dates:
                row += self.attendance[name].get(d, "-").center(7)
            row += f"{self.percentage(name):7.2f}%"
            print(row)

    def display_summary(self, threshold=DEFAULT_THRESHOLD):
        print(f"{'Student':<14}{'P':>3}{'L':>3}{'A':>3}{'Total':>7}{'Att%':>9}  Status")
        print("-" * 52)
        for name in self.students:
            p, l, a = self.counts(name)
            pct = self.percentage(name)
            flag = "OK" if pct >= threshold else "LOW - SHORTAGE"
            print(f"{name:<14}{p:>3}{l:>3}{a:>3}{p + l + a:>7}{pct:>8.2f}%  {flag}")
        low = self.below_threshold(threshold)
        print(f"\nStudents below {threshold:.0f}%: ", end="")
        print(", ".join(f"{n} ({p}%)" for n, p in low) if low else "None")

    def class_average(self):
        if not self.students:
            return 0.0
        return round(sum(self.percentage(s) for s in self.students) / len(self.students), 2)


# ------------------------------------------------------------------
# Interactive menu
# ------------------------------------------------------------------
def menu():
    tracker = AttendanceTracker()
    options = """
===== ATTENDANCE TRACKER =====
1. Add student
2. Add class date
3. Mark / update attendance
4. Display attendance records
5. Show attendance summary
6. Students below threshold
7. Exit
"""
    while True:
        print(options)
        choice = input("Enter choice: ").strip()
        try:
            if choice == "1":
                print("Added:", tracker.add_student(input("Student name: ")))
            elif choice == "2":
                tracker.add_class_date(input("Date (e.g. 01-Oct): "))
                print("Class date added.")
            elif choice == "3":
                n = input("Student name: ")
                d = input("Date: ")
                s = input("Status (P/A/L): ")
                print("Record", tracker.mark_attendance(n, d, s))
            elif choice == "4":
```

```
                tracker.display_records()
            elif choice == "5":
                tracker.display_summary()
            elif choice == "6":
                t = float(input("Threshold %: "))
                for n, p in tracker.below_threshold(t):
                    print(f"{n}: {p}%")
            elif choice == "7":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except (ValueError, KeyError) as err:
            print("Error:", err)


if __name__ == "__main__":
    menu()
```

A.4 Test program: test_attendance.py

```python
from attendance_tracker import AttendanceTracker


def banner(title):
    print("=" * 60)
    print(title)
    print("=" * 60)


def build_class(dates, names):
    t = AttendanceTracker()
    for n in names:
        t.add_student(n)
    for d in dates:
        t.add_class_date(d)
    return t


# ---------------- Scenario 1: mixed attendance ----------------
banner("SCENARIO 1: Mixed attendance (6 students, 8 classes)")
dates = ["01-Oct", "03-Oct", "07-Oct", "09-Oct", "14-Oct", "16-Oct", "21-Oct", "23-Oct"]
t = build_class(dates, ["Aarav", "Diya", "Rohan", "Meera", "Karthik", "Sneha"])
pattern = {
    "Aarav":   "PPPPPPPP",
    "Diya":    "PPPLPPPP",
    "Rohan":   "PAPAPAPP",
    "Meera":   "PPAAPPAP",
    "Karthik": "AAPAPAAP",
    "Sneha":   "PPPPPAPP",
}
for name, codes in pattern.items():
    for d, c in zip(dates, codes):
        t.mark_attendance(name, d, c)
t.display_records()
print()
t.display_summary(75)
print("\nClass average:", t.class_average(), "%")

# ---------------- Scenario 2: updating records ----------------
banner("SCENARIO 2: Updating a wrong entry")
print("Karthik on 03-Oct ->", t.mark_attendance("Karthik", "03-Oct", "P"))
print("Rohan on 21-Oct   ->", t.mark_attendance("rohan", "21-Oct", "L"))
print(f"Karthik now: {t.percentage('Karthik')}%   Rohan now: {t.percentage('Rohan')}%")

# ---------------- Scenario 3: different thresholds ----------------
banner("SCENARIO 3: Threshold comparison")
for th in (60, 75, 85):
    low = t.below_threshold(th)
    print(f"Below {th}%:", [(n, p) for n, p in low] or "None")

# ---------------- Scenario 4: everyone perfect ----------------
banner("SCENARIO 4: Perfect attendance")
t2 = build_class(["01-Oct", "02-Oct", "03-Oct"], ["Ishaan", "Priya"])
for n in t2.students:
    for d in t2.class_dates:
        t2.mark_attendance(n, d, "P")
t2.display_summary(75)

# ---------------- Scenario 5: exactly on the threshold ----------------
banner("SCENARIO 5: Boundary value (exactly 75%)")
t3 = build_class(["D1", "D2", "D3", "D4"], ["Neha"])
for d, c in zip(t3.class_dates, "PPPA"):
    t3.mark_attendance("Neha", d, c)
print("Neha:", t3.percentage("Neha"), "%")
print("Below 75% ->", t3.below_threshold(75) or "None (75 is not below 75)")
print("Below 76% ->", t3.below_threshold(76))

# ---------------- Scenario 6: student added mid-term ----------------
banner("SCENARIO 6: New student added mid-term (no records yet)")
t4 = build_class(["D1", "D2"], ["Arjun"])
t4.mark_attendance("Arjun", "D1", "P")
t4.mark_attendance("Arjun", "D2", "P")
t4.add_student("Zoya")
```

```
t4.display_records()
print("Zoya %:", t4.percentage("Zoya"), "(no classes recorded, so 0.0)")

# ---------------- Scenario 7: invalid inputs ----------------
banner("SCENARIO 7: Error handling")
tests = [
    ("Duplicate student", lambda: t.add_student("aarav")),
    ("Empty name", lambda: t.add_student("   ")),
    ("Unknown student", lambda: t.mark_attendance("Ghost", "01-Oct", "P")),
    ("Unknown date", lambda: t.mark_attendance("Aarav", "30-Dec", "P")),
    ("Bad status", lambda: t.mark_attendance("Aarav", "01-Oct", "X")),
    ("Duplicate date", lambda: t.add_class_date("01-Oct")),
]
for label, action in tests:
    try:
        action()
        print(f"{label:<18}: NO ERROR (unexpected)")
    except (ValueError, KeyError) as e:
        print(f"{label:<18}: caught -> {e}")

# ---------------- Scenario 8: empty tracker ----------------
banner("SCENARIO 8: Empty tracker")
AttendanceTracker().display_records()

# ---------------- Assertions ----------------
assert t.percentage("Aarav") == 100.0
assert t3.percentage("Neha") == 75.0
assert t4.percentage("Zoya") == 0.0
print("\nAll assertions passed.")
```

A.5 Test scenarios, output and interpretation

Each block below is the genuine terminal output of the test program.

Scenario 1: Mixed Attendance (6 Students, 8 Classes)

```
Student   01-Oct 03-Oct 07-Oct 09-Oct 14-Oct 16-Oct 21-Oct 23-Oct    Att%
------------------------------------------------------------------------
Aarav       P      P      P      P      P      P      P      P    100.00%
Diya        P      P      P      L      P      P      P      P    100.00%
Rohan       P      A      P      A      P      A      P      P     62.50%
Meera       P      P      A      A      P      P      A      P     62.50%
Karthik     A      A      P      A      P      A      A      P     37.50%
Sneha       P      P      P      P      P      A      P      P     87.50%

Student         P  L  A  Total     Att%  Status
-------------------------------------------------
Aarav           8  0  0      8  100.00%  OK
Diya            7  1  0      8  100.00%  OK
Rohan           5  0  3      8   62.50%  LOW - SHORTAGE
Meera           5  0  3      8   62.50%  LOW - SHORTAGE
Karthik         3  0  5      8   37.50%  LOW - SHORTAGE
Sneha           7  0  1      8   87.50%  OK

Students below 75%: Karthik (37.5%), Rohan (62.5%), Meera (62.5%)

Class average: 75.0 %
```

Interpretation: Aarav and Diya are at 100% (Diya's single late arrival still counts as attended). Rohan and Meera sit at 62.5%, Karthik at 37.5% and Sneha at 87.5%. Three students fall below 75% and are flagged with 'LOW - SHORTAGE'. The class average works out to exactly 75.0%.

Scenario 2: Updating A Wrong Entry

```
Karthik on 03-Oct -> updated (A -> P)
Rohan on 21-Oct   -> updated (P -> L)
Karthik now: 50.0%   Rohan now: 62.5%
```

Interpretation: Karthik's 03-Oct entry was corrected from A to P, so his percentage rises from 37.5% to 50%. Rohan's 21-Oct entry changed from P to L; his percentage stays at 62.5% because Late counts as attended. The lower-case name 'rohan' still found the right student.

Scenario 3: Threshold Comparison

```
Below 60%: [('Karthik', 50.0)]
Below 75%: [('Karthik', 50.0), ('Rohan', 62.5), ('Meera', 62.5)]
Below 85%: [('Karthik', 50.0), ('Rohan', 62.5), ('Meera', 62.5)]
```

Interpretation: At a 60% threshold only Karthik is flagged. At 75% and 85% the same three students are flagged (Karthik 50%, Rohan 62.5%, Meera 62.5%) because Sneha at 87.5% clears both limits. This shows the threshold is a parameter and not hard-coded.

Scenario 4: Perfect Attendance

```
Student         P  L  A  Total     Att%  Status
-------------------------------------------------
Ishaan          3  0  0      3  100.00%  OK
Priya           3  0  0      3  100.00%  OK

Students below 75%: None
```

Interpretation: When every student attends every class, the below-threshold list is empty and the program prints 'None' instead of failing.

Scenario 5: Boundary Value (Exactly 75%)

```
Neha: 75.0 %
Below 75% -> None (75 is not below 75)
Below 76% -> [('Neha', 75.0)]
```

Interpretation: Boundary-value test: 3 present out of 4 is exactly 75%. Because the rule is 'strictly below', Neha is NOT flagged at 75, but she is flagged at 76.

Scenario 6: New Student Added Mid-Term (No Records Yet)

```
Student     D1     D2      Att%
------------------------------
Arjun       P      P     100.00%
Zoya        -      -       0.00%
Zoya %: 0.0 (no classes recorded, so 0.0)
```

Interpretation: Zoya was added after attendance had started and has no records. The grid shows '-' for her dates and her percentage is 0.0 rather than a divide-by-zero error.

Scenario 7: Error Handling

```
Duplicate student : caught -> Student 'Aarav' already exists.
Empty name        : caught -> Student name cannot be empty.
Unknown student   : caught -> "Student 'Ghost' not found."
Unknown date      : caught -> "Date '30-Dec' is not a recorded class date."
Bad status        : caught -> Status must be P, A or L.
Duplicate date    : caught -> Class date '01-Oct' already exists.
```

Interpretation: Six invalid operations were tried (duplicate student, empty name, unknown student, unknown date, bad status, duplicate date). All were caught with clear messages.

Scenario 8: Empty Tracker

```
No students recorded yet.
```

Interpretation: An empty tracker prints a friendly message instead of an empty or broken table.

Result: all assertions passed.

Part B: Sales Data Analyzer

Store product sales, calculate revenue, find the best seller and compare performance.

B.1 Approach

Every product is a key in a dictionary whose value holds its **price** and the **quantity** sold. A list preserves product order. Revenue is never stored; it is computed from price x quantity on demand, so updating a price or quantity can never leave stale totals.

Structure	Type	Purpose
products	list	Ordered product names (display order)
sales	dict of dicts	{name: {'price': float, 'quantity': int}}

Formulas

- **Product revenue** = price x quantity sold.
- **Total sales** = sum of all product revenues.
- **Revenue share** = product revenue / total sales x 100.
- **Highest selling** is reported two ways: by *units sold* and by *revenue*; ties are all returned.
- **Performance label**: 'Top performer' if revenue is at least 125% of the mean revenue, 'Low performer' if below 75%, otherwise 'Average'.

B.2 Methods implemented

Method	What it does
add_product(name, price, quantity)	Adds a product; validates name, price (> 0) and quantity (>= 0)
record_sale(name, quantity)	Adds newly sold units to an existing product
set_quantity(name, quantity)	Overwrites quantity to correct a wrong entry
update_price(name, price)	Changes the unit price
revenue / revenue_by_product / total_sales	Per-product revenue, dictionary of all revenues, grand total
best_selling_by_quantity / best_by_revenue	Top product(s) by units and by revenue, tie-safe
ranking()	Products sorted by revenue, highest first
compare(a, b)	Tells which of two products earned more and by how much
performance_label(name)	Top / Average / Low performer versus the mean
display_table() / summary()	Formatted table with totals, shares, best sellers and ranking

B.3 Source code: sales_analyzer.py

```
"""
Sales Data Analyzer  -  Python Programming (Unit 3 Mini Project)
Author : Sriram Satvik Karanam | USN : 25BTRAD049 | CSE-AIDE B

Data structures used
--------------------
products : list  -> ordered list of product names
sales    : dict  -> {product_name: {"price": float, "quantity": int}}
"""
```

```python
class SalesAnalyzer:
    def __init__(self):
        self.products = []   # list of names
        self.sales = {}      # dict of dicts

    # ------------------------------------------------------------------
    # 1. Adding / updating records
    # ------------------------------------------------------------------
    def add_product(self, name, price, quantity=0):
        name = name.strip().title()
        if not name:
            raise ValueError("Product name cannot be empty.")
        if name in self.sales:
            raise ValueError(f"Product '{name}' already exists.")
        if price <= 0:
            raise ValueError("Price must be greater than zero.")
        if quantity < 0:
            raise ValueError("Quantity cannot be negative.")
        self.products.append(name)
        self.sales[name] = {"price": float(price), "quantity": int(quantity)}

    def record_sale(self, name, quantity):
        """Add units sold to an existing product."""
        name = name.strip().title()
        if name not in self.sales:
            raise KeyError(f"Product '{name}' not found.")
        if quantity <= 0:
            raise ValueError("Quantity sold must be positive.")
        self.sales[name]["quantity"] += int(quantity)

    def set_quantity(self, name, quantity):
        """Overwrite the quantity (used to correct a wrong entry)."""
        name = name.strip().title()
        if name not in self.sales:
            raise KeyError(f"Product '{name}' not found.")
        if quantity < 0:
            raise ValueError("Quantity cannot be negative.")
        self.sales[name]["quantity"] = int(quantity)

    def update_price(self, name, price):
        name = name.strip().title()
        if name not in self.sales:
            raise KeyError(f"Product '{name}' not found.")
        if price <= 0:
            raise ValueError("Price must be greater than zero.")
        self.sales[name]["price"] = float(price)

    # ------------------------------------------------------------------
    # 2. Calculations
    # ------------------------------------------------------------------
    def revenue(self, name):
        item = self.sales[name]
        return item["price"] * item["quantity"]

    def revenue_by_product(self):
        return {name: self.revenue(name) for name in self.products}

    def total_sales(self):
        return sum(self.revenue_by_product().values())

    def total_units(self):
        return sum(item["quantity"] for item in self.sales.values())

    def best_selling_by_quantity(self):
        """Return list of product(s) with the highest units sold (handles ties)."""
        if not self.products:
            return []
        top = max(self.sales[p]["quantity"] for p in self.products)
        return [p for p in self.products if self.sales[p]["quantity"] == top]

    def best_by_revenue(self):
        if not self.products:
            return []
        rev = self.revenue_by_product()
        top = max(rev.values())
```

```
        return [p for p in self.products if rev[p] == top]

    def ranking(self):
        """Products sorted by revenue, highest first."""
        rev = self.revenue_by_product()
        return sorted(rev.items(), key=lambda kv: kv[1], reverse=True)

    def share_percent(self, name):
        total = self.total_sales()
        return round(self.revenue(name) / total * 100, 2) if total else 0.0

    def compare(self, a, b):
        a, b = a.strip().title(), b.strip().title()
        for n in (a, b):
            if n not in self.sales:
                raise KeyError(f"Product '{n}' not found.")
        ra, rb = self.revenue(a), self.revenue(b)
        if ra == rb:
            verdict = f"{a} and {b} earned the same revenue."
        else:
            win, lose = (a, b) if ra > rb else (b, a)
            diff = abs(ra - rb)
            verdict = f"{win} out-earned {lose} by Rs. {diff:,.2f}."
        return verdict

    def performance_label(self, name):
        """Top / Average / Low based on comparison with mean revenue."""
        if not self.products:
            return "N/A"
        mean = self.total_sales() / len(self.products)
        r = self.revenue(name)
        if r >= 1.25 * mean:
            return "Top performer"
        if r < 0.75 * mean:
            return "Low performer"
        return "Average"

    # ------------------------------------------------------------------
    # 3. Display
    # ------------------------------------------------------------------
    def display_table(self):
        if not self.products:
            print("No products recorded yet.")
            return
        w = max(len("Product"), max(len(p) for p in self.products)) + 2
        head = f"{'Product':<{w}}{'Price':>10}{'Qty':>7}{'Revenue':>14}{'Share':>9}"
        print(head)
        print("-" * len(head))
        for p in self.products:
            item = self.sales[p]
            print(f"{p:<{w}}{item['price']:>10,.2f}{item['quantity']:>7}"
                  f"{self.revenue(p):>14,.2f}{self.share_percent(p):>8.2f}%")
        print("-" * len(head))
        print(f"{'TOTAL':<{w}}{'':>10}{self.total_units():>7}{self.total_sales():>14,.2f}{'100.00%':>9}")

    def summary(self):
        if not self.products:
            print("No data to summarise.")
            return
        self.display_table()
        print()
        print("Highest selling (units)  :", ", ".join(self.best_selling_by_quantity()),
              f"({self.sales[self.best_selling_by_quantity()[0]]['quantity']} units)")
        print("Highest revenue          :", ", ".join(self.best_by_revenue()),
              f"(Rs. {self.revenue(self.best_by_revenue()[0]):,.2f})")
        print("\nRanking by revenue:")
        for rank, (p, r) in enumerate(self.ranking(), start=1):
            print(f"  {rank}. {p:<14} Rs. {r:>12,.2f}   [{self.performance_label(p)}]")


# ------------------------------------------------------------------
# Interactive menu
# ------------------------------------------------------------------
def menu():
    app = SalesAnalyzer()
    options = """
```

```
===== SALES DATA ANALYZER =====
1. Add product
2. Record sale (add units)
3. Correct quantity
4. Update price
5. Show sales table
6. Full summary
7. Compare two products
8. Exit
"""
    while True:
        print(options)
        choice = input("Enter choice: ").strip()
        try:
            if choice == "1":
                app.add_product(input("Name: "), float(input("Price: ")), int(input("Qty sold: ")))
            elif choice == "2":
                app.record_sale(input("Name: "), int(input("Units sold: ")))
            elif choice == "3":
                app.set_quantity(input("Name: "), int(input("New quantity: ")))
            elif choice == "4":
                app.update_price(input("Name: "), float(input("New price: ")))
            elif choice == "5":
                app.display_table()
            elif choice == "6":
                app.summary()
            elif choice == "7":
                print(app.compare(input("First product: "), input("Second product: ")))
            elif choice == "8":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except (ValueError, KeyError) as err:
            print("Error:", err)


if __name__ == "__main__":
    menu()
```

B.4 Test program: test_sales.py

```python
from sales_analyzer import SalesAnalyzer


def banner(title):
    print("=" * 62)
    print(title)
    print("=" * 62)


def load(data):
    a = SalesAnalyzer()
    for name, price, qty in data:
        a.add_product(name, price, qty)
    return a


# ---------------- Dataset 1: electronics store ----------------
banner("DATASET 1: Electronics store")
shop = load([
    ("Laptop", 55000, 12),
    ("Smartphone", 22000, 30),
    ("Headphones", 2500, 85),
    ("Keyboard", 1200, 60),
    ("Smartwatch", 8000, 18),
])
shop.summary()

# ---------------- Updating records ----------------
banner("UPDATING RECORDS: new sales, correction and price change")
shop.record_sale("laptop", 3)
shop.set_quantity("Keyboard", 75)
shop.update_price("Smartwatch", 7500)
shop.display_table()
print()
print("Compare Laptop vs Smartphone:", shop.compare("Laptop", "Smartphone"))
print("Compare Headphones vs Keyboard:", shop.compare("Headphones", "Keyboard"))
print("Best by units  :", shop.best_selling_by_quantity())
print("Best by revenue:", shop.best_by_revenue())

# ---------------- Dataset 2: grocery ----------------
banner("DATASET 2: Grocery store (low prices, high volume)")
grocery = load([
    ("Rice 1kg", 60, 400),
    ("Milk 1L", 54, 520),
    ("Bread", 40, 310),
    ("Olive Oil", 650, 25),
    ("Eggs Dozen", 84, 200),
])
grocery.summary()

# ---------------- Dataset 3: tie in sales ----------------
banner("DATASET 3: Tie in units sold")
tie = load([("Pen", 10, 100), ("Notebook", 50, 100), ("Marker", 30, 40)])
tie.display_table()
print("Best by units  :", tie.best_selling_by_quantity(), "<- tie handled")
print("Best by revenue:", tie.best_by_revenue())

# ---------------- Dataset 4: zero sales ----------------
banner("DATASET 4: Product with zero sales")
zero = load([("Tablet", 30000, 0), ("Charger", 800, 15)])
zero.display_table()
print("Tablet label :", zero.performance_label("Tablet"))
print("Charger label:", zero.performance_label("Charger"))

# ---------------- Error handling ----------------
banner("ERROR HANDLING")
tests = [
    ("Duplicate product", lambda: shop.add_product("laptop", 100, 1)),
    ("Negative price", lambda: shop.add_product("Mouse", -5, 1)),
    ("Negative quantity", lambda: shop.add_product("Mouse", 500, -1)),
    ("Unknown product", lambda: shop.record_sale("Drone", 2)),
    ("Zero units sold", lambda: shop.record_sale("Laptop", 0)),
    ("Compare unknown", lambda: shop.compare("Laptop", "Drone")),
```

```
]
for label, action in tests:
    try:
        action()
        print(f"{label:<18}: NO ERROR (unexpected)")
    except (ValueError, KeyError) as e:
        print(f"{label:<18}: caught -> {e}")

# ---------------- Empty analyzer ----------------
banner("EMPTY ANALYZER")
empty = SalesAnalyzer()
empty.summary()
print("Total sales:", empty.total_sales(), "| Best:", empty.best_selling_by_quantity())

# ---------------- Assertions ----------------
assert load([("A", 10, 5)]).total_sales() == 50
assert shop.best_selling_by_quantity() == ["Headphones"]
assert sorted(tie.best_selling_by_quantity()) == ["Notebook", "Pen"]
assert tie.best_by_revenue() == ["Notebook"]
print("\nAll assertions passed.")
```

B.5 Test datasets, output and interpretation

Prices are in rupees. Outputs are copied from the real run of test_sales.py.

Dataset 1: Electronics Store

```
Product          Price    Qty       Revenue    Share
--------------------------------------------------
Laptop       55,000.00     12    660,000.00   37.75%
Smartphone   22,000.00     30    660,000.00   37.75%
Headphones    2,500.00     85    212,500.00   12.15%
Keyboard      1,200.00     60     72,000.00    4.12%
Smartwatch    8,000.00     18    144,000.00    8.24%
--------------------------------------------------
TOTAL                     205  1,748,500.00  100.00%

Highest selling (units)  : Headphones (85 units)
Highest revenue          : Laptop, Smartphone (Rs. 660,000.00)

Ranking by revenue:
  1. Laptop         Rs.   660,000.00   [Top performer]
  2. Smartphone     Rs.   660,000.00   [Top performer]
  3. Headphones     Rs.   212,500.00   [Low performer]
  4. Smartwatch     Rs.   144,000.00   [Low performer]
  5. Keyboard       Rs.    72,000.00   [Low performer]
```

Interpretation: Laptop and Smartphone tie for the highest revenue at Rs. 660,000 each (37.75% of the Rs. 1,748,500 total), yet **Headphones** is the highest-selling product by units (85). So 'best seller' depends on the measure: Headphones wins on volume, Laptop and Smartphone win on money. Note that the mean revenue is about Rs. 349,700, so Headphones (Rs. 212,500) falls under the 75% cut-off and is labelled 'Low performer' on revenue, even though it leads in units.

Updating Records: New Sales, Correction And Price Change

```
Product          Price    Qty       Revenue    Share
--------------------------------------------------
Laptop       55,000.00     15    825,000.00   42.91%
Smartphone   22,000.00     30    660,000.00   34.33%
Headphones    2,500.00     85    212,500.00   11.05%
Keyboard      1,200.00     75     90,000.00    4.68%
Smartwatch    7,500.00     18    135,000.00    7.02%
--------------------------------------------------
TOTAL                     223  1,922,500.00  100.00%

Compare Laptop vs Smartphone: Laptop out-earned Smartphone by Rs. 165,000.00.
Compare Headphones vs Keyboard: Headphones out-earned Keyboard by Rs. 122,500.00.
Best by units  : ['Headphones']
Best by revenue: ['Laptop']
```

Interpretation: After 3 more laptops were sold, the Keyboard quantity corrected to 75 and the Smartwatch price cut to Rs. 7,500, the total rose to Rs. 1,922,500. Laptop (15 units) now clearly leads revenue by Rs. 165,000 over Smartphone, while Headphones still leads by units. The comparisons confirm this.

Dataset 2: Grocery Store (Low Prices, High Volume)

```
Product          Price    Qty       Revenue    Share
--------------------------------------------------
Rice 1Kg         60.00    400     24,000.00   24.61%
Milk 1L          54.00    520     28,080.00   28.79%
Bread            40.00    310     12,400.00   12.71%
Olive Oil       650.00     25     16,250.00   16.66%
Eggs Dozen       84.00    200     16,800.00   17.23%
--------------------------------------------------
TOTAL                    1455     97,530.00  100.00%

Highest selling (units)  : Milk 1L (520 units)
Highest revenue          : Milk 1L (Rs. 28,080.00)

Ranking by revenue:
  1. Milk 1L        Rs.    28,080.00   [Top performer]
  2. Rice 1Kg       Rs.    24,000.00   [Average]
  3. Eggs Dozen     Rs.    16,800.00   [Average]
  4. Olive Oil      Rs.    16,250.00   [Average]
  5. Bread          Rs.    12,400.00   [Low performer]
```

Interpretation: In a high-volume, low-price store, Milk 1L leads on both units (520) and revenue (Rs. 28,080). Olive Oil sold only 25 units, yet its revenue (Rs. 16,250) beats Bread (Rs. 12,400) which sold 310 units, showing how a high price compensates for low volume.

Dataset 3: Tie In Units Sold

```
Product        Price    Qty       Revenue    Share
--------------------------------------------------
Pen            10.00    100      1,000.00   13.89%
Notebook       50.00    100      5,000.00   69.44%
Marker         30.00     40      1,200.00   16.67%
--------------------------------------------------
TOTAL                   240      7,200.00  100.00%
Best by units  : ['Pen', 'Notebook'] <- tie handled
Best by revenue: ['Notebook']
```

Interpretation: Pen and Notebook both sold 100 units, so both are returned as best sellers by quantity (tie handled correctly). By revenue, Notebook clearly wins (Rs. 5,000 vs Rs. 1,000) because of its higher price.

Dataset 4: Product With Zero Sales

```
Product       Price    Qty       Revenue    Share
--------------------------------------------------
Tablet    30,000.00      0          0.00    0.00%
Charger      800.00     15     12,000.00  100.00%
--------------------------------------------------
TOTAL                   15     12,000.00  100.00%
Tablet label : Low performer
Charger label: Top performer
```

Interpretation: Tablet has a good price but zero sales, so its revenue is 0 and it is labelled 'Low performer'. Charger holds 100% of the revenue. No division errors occur.

Error Handling

```
Duplicate product : caught -> Product 'Laptop' already exists.
Negative price    : caught -> Price must be greater than zero.
Negative quantity : caught -> Quantity cannot be negative.
Unknown product   : caught -> "Product 'Drone' not found."
Zero units sold   : caught -> Quantity sold must be positive.
Compare unknown   : caught -> "Product 'Drone' not found."
```

Interpretation: Six invalid operations were attempted (duplicate product, negative price, negative quantity, unknown product, zero units, comparing an unknown product). All were rejected with clear messages.

Empty Analyzer

```
No data to summarise.
Total sales: 0 | Best: []
```

Interpretation: An analyzer with no data prints a message, returns total sales of 0 and an empty best-seller list instead of crashing.

Result: all assertions passed.

Part C: Conclusion

Both programs meet every expected functionality in the problem statements.

Requirement checklist

Requirement	Attendance Tracker	Sales Data Analyzer
Uses lists and dictionaries	students, class_dates (lists); attendance (dict of dicts)	products (list); sales (dict of dicts)
Add records	add_student, add_class_date	add_product
Update records	mark_attendance (add or update)	record_sale, set_quantity, update_price
Organised display	display_records, display_summary	display_table, summary
Key calculations	Percentage, class average, below-threshold list	Revenue, total sales, share %, ranking
Identify best / flagged	Students below a given threshold	Highest seller by units and by revenue
Testing	8 scenarios incl. boundary and errors	4 datasets plus update, error, empty tests

What was learned

- A dictionary of dictionaries is a natural way to model records such as student-date-status or product-price-quantity.
- Computing derived values (percentage, revenue) on demand avoids inconsistent data after updates.
- Boundary tests (exactly 75%) and empty-data tests catch bugs that normal inputs never reveal.
- Input validation with `ValueError` and `KeyError` keeps the menu programs from crashing.
- The same measure can rank differently: units sold and revenue gave different 'best' products in the sales data.

Possible improvements

- Save and load data from CSV or JSON files so records persist between runs.
- Add dates to sales to study monthly trends and plot charts.
- Export the attendance shortage list to a file to share with the faculty.

How to run

```
python attendance_tracker.py     # interactive menu
python sales_analyzer.py         # interactive menu
python test_attendance.py        # all attendance scenarios
python test_sales.py             # all sales datasets
```